Year II — Computational Mathematics

Based on lectures by Various Lecturers

Notes taken by Anonymous

Michaelmas 2026

Contents

1 Numerical and symbolic Computing **2**
1.1 Fixed-point numbers . . . 2
1.2 Floating-point numbers . . . 2
1.3 Significant Figures . . . 4
1.4 Rounding Error . . . 4
1.5 Loss of Significance . . . 5
1.6 Symbolic Vs Numerical Representations . . . 5

2 Continuous Functions **6**
2.1 Interpolation . . . 6
2.1.1 Motivation and Taylor Approximation . . . 6
2.1.2 Polynomial Interpolation . . . 6
2.1.3 Lagrange Interpolation . . . 7
2.1.4 Interpolation Error . . . 8
2.1.5 Runge Phenomenon and Chebyshev Nodes . . . 8
2.2 Non-Linear Equations . . . 9
2.2.1 Problem and Motivation . . . 9
2.2.2 Bisection Method . . . 9
2.2.3 Fixed-Point Iteration . . . 10
2.2.4 Order of Convergence . . . 11
2.2.5 Newton's Method . . . 11
2.2.6 Newton's Method for Systems . . . 12
2.2.7 Secant Method . . . 13
2.2.8 Comparison of Root-Finding Methods . . . 13
2.2.9 Problem-Solving Checklist . . . 13

Chapter 1

Numerical and symbolic Computing

1.1 Fixed-point numbers

We see that when we attempt $0.1 + 0.2 == 0.3$ in MATLAB, we return a false value due to rounding in binary causing issues. Numbers like 0.1 and 0.2 cannot be stored in binary exactly and thus the two sides will differ by 5.6×10^{-17}. We utilise real numbers and integers within computational systems to speed up manipulating mathematical objects. We can do this symbolically with *Computer Algebra Systems* but we trade off time.

Definition 1.1.1 (Fixed-point numbers)**.** A *fixed-point number* representation of a real number is given as:

$$x = \pm \left(d_1 d_2 \cdots d_{k-1} \cdot d_k \cdots d_n\right)_\beta, \quad \text{where} \quad d_1, \ldots, d_n \in \{0, 1, \ldots, \beta - 1\}.$$

Here β is the base (e.g. 10 for decimal arithmetic or 2 for binary). The word fixed refers to the position of the point: a fixed-point system uses a fixed number of digits before and after the point, so the numbers it can represent are equally spaced, a distance $\beta^{-(n-k+1)}$ apart.

Remark 1.1.2. This system can be used to represent any real number uniquely. We must remove representations ending in infinitely many 9's (when in decimal) or infinitely many $b-1$'s where b represents the base. For example in denary we cannot use $3.14 = 3.1400\ldots = 3.139999\ldots$ as we are excluding the $10-1$'s expansion.

1.2 Floating-point numbers

Definition 1.2.1 (Floating-point numbers)**.** Fix a base $\beta \geq 2$, a number of digits m, and an exponent range $e_{\min} \leq e \leq e_{\max}$. Only numbers in the floating-point number system $F \subset \mathbb{R}$ can be represented exactly, where

$$F = \left\{ \pm \left(0.d_1 d_2 \cdots d_m\right)_\beta \beta^e \mid d_i \in \{0, 1, \ldots, \beta - 1\}, e \in \mathbb{Z}, e_{\min} \leq e \leq e_{\max} \right\}$$

Here $\left(0.d_1 d_2 \cdots d_m\right)_\beta$ is called the fraction (or significand or mantissa), and e is the exponent.

Example 1.2.2. Suppose we set $\beta = 10$ and $m = 3$ and:

$$-2 \leq e \leq 2.$$

Then we could represent

$$0.123 \times 10^2 = 12.3$$

or

$$0.456 \times 10^{-2} = 0.00456.$$

We could also represent

$$-0.789 \times 10^{1} = -7.89.$$

But something like

$$12.3456$$

cannot be represented exactly, because we only have 3 digits available for the significand.

Example 1.2.3. Now consider a system with $\beta = 2$ so we are working in binary. We set $m = 3$ and once more we have:

$$-2 \leq e \leq 2.$$

We now represent numbers such as:

$$(0.101)_2 2^2 = (\frac{1}{2} + \frac{1}{8}) \times 4 = \frac{5}{8} \times 4 = 2.5$$

or something like

$$(0.100)_2 2^1 = (\frac{1}{8}) \times 2 = 0.25$$

Definition 1.2.4 (Normalisation)**.** We use normalisation, enforcing the requirement $d_1 \neq 0$ to create a unique numbering system and so we assume that our system is *normalised* by default. This is useful as we can represent a larger range of numbers at the loss of equally spaced numbers.

Example 1.2.5. Here $\beta = 2$, and there are 52 bits for the fraction, 11 for the exponent, and 1 for the sign. The stored exponent field is $E = (e_1 e_2 \cdots e_{11})_2 \in [0, 2047]$, and a normalised number has the value

$$\pm\,(1.d_1 \cdots d_{52})_2\, 2^{E-1023} = \pm\,(0.1d_1 \cdots d_{52})_2\, 2^{E-1022}.$$

When $\beta = 2$, the first digit of a normalised number is always 1, so it doesn't need to be stored in memory. The standard's **exponent bias** is 1023; in the $(0.1 \cdots)_2$ form used in these notes it becomes 1022, so the exponents $e = E - 1022$ lie in the range -1022 to 1025. The two extreme values of the exponent field are reserved: $E = 0$ stores ± 0 and the subnormal numbers (see below), and $E = 2047$ stores $\pm\infty$ and NaN ("not a number"). Normalised numbers therefore have exponents from -1021 to 1024.

Remark 1.2.6. The smallest positive normalised number in this system is $(0.1)_2 2^{-1021} \approx 2.225 \times 10^{-308}$, and the largest number is $(0.1 \cdots 1)_2 2^{1024} \approx 1.798 \times 10^{308}$. In MATLAB, these are returned by realmin and realmax.

Definition 1.2.7 (Over and Under Flow)**.** Numbers outside the finite set F cannot be represented exactly. If a calculation produces a non-zero result that is smaller in absolute value than the smallest normalised number, it is called **underflow**; IEEE arithmetic first represents such results using subnormal numbers (see below), and results smaller still are rounded to $\boldsymbol{o}$. If a result is larger in absolute value than the largest number in F, it is called **overflow**; in IEEE arithmetic (and hence in MATLAB), the result is $\pm\infty$, so that 2^* realmax returns 'Inf'.

Definition 1.2.8 (Rounding)**.** The mapping from $\mathbb{R}$ to F is called rounding and denoted $\mathrm{fl}(x)$. Usually it is simply the nearest number in F to x. If x lies exactly midway between two numbers in F, a method of breaking ties is required. The IEEE standard specifies round to nearest even-i.e., take the neighbour with last digit o in the fraction. Ties are then rounded up and down equally often, which avoids statistical bias or prolonged drift in one direction.

1.3 Significant Figures

Definition 1.3.1 (Signficant Figures)**.** To count the number of significant figures in a number x, start with the first non-zero digit from the left, and count all the digits thereafter, including final zeros if they are after the decimal point.

1.4 Rounding Error

Definition 1.4.1 (Absolute and Relative Errors)**.** If x is the true value of a number and $\hat{x}$ is an approximation, we define the absolute error of this approximation as the quantity

$$|\hat{x} - x|,$$

while the relative error of the approximation is given by

$$\frac{|\hat{x} - x|}{|x|},$$

assuming $x \neq 0$.

Remark 1.4.2. Relative errors are often more useful because they are scale invariant

Definition 1.4.3 (Relative Error in Rounding)**.** If $|x|$ lies between the smallest non-zero number in F and the largest number in F, then we can write

$$\mathrm{fl}(x) = x(1+\delta), \qquad \text{where} \qquad \delta = \frac{\mathrm{fl}(x) - x}{x},$$

so that $|\delta|$ is the relative error incurred by rounding.

Definition 1.4.4 (Unit Round Off)**.** To bound $|\delta|$, write $x = (0.d_1 d_2 \cdots)_\beta \beta^e$ with $d_1 \neq 0$ and $e \in [e_{\min}, e_{\max}]$. If $x \notin F$, this significand will not terminate after m digits, but rounding changes it by at most $\frac{1}{2}\beta^{-m}$, so

$$|\mathrm{fl}(x) - x| \leq \frac{1}{2}\beta^{-m}\beta^e.$$

Since $d_1 \neq 0$, the significand of x is at least $(0.1)_\beta = \beta^{-1}$, so $|x| \geq \beta^{-1}\beta^e$. Therefore

$$|\delta| = \frac{|\mathrm{fl}(x) - x|}{|x|} \leq \frac{\frac{1}{2}\beta^{-m}\beta^e}{\beta^{-1}\beta^e} = \frac{1}{2}\beta^{1-m}.$$

The number $\epsilon_{\mathrm{M}} = \frac{1}{2}\beta^{1-m}$ is called the unit roundoff, and is independent of x. So the relative rounding error satisfies

$$|\delta| \leq \epsilon_{\mathrm{M}}.$$

Remark 1.4.5. For $\circ = +, -, \times, \div$, IEEE standard arithmetic requires correctly rounded operations: the result must be the exact result rounded to F. Provided no overflow or underflow occurs, this means that

$$\mathrm{fl}(x \circ y) = (x \circ y)(1+\delta), \quad |\delta| \leq \epsilon_{\mathrm{M}}.$$

1.5 Loss of Significance

Definition 1.5.1 (Loss of Significance)**.** You might think that the above guarantees the accuracy of calculations to within ϵ_{M}, but this is true only if x and y are themselves exact. In reality, we are probably starting from $\bar{x} = x(1+\delta_1)$ and $\bar{y} = y(1+\delta_2)$, with $|\delta_1|, |\delta_2| \leq \epsilon_{\mathrm{M}}$. In that case, there is an error even before we round the result, since

$$\begin{aligned} \bar{x} \pm \bar{y} &= x(1+\delta_1) \pm y(1+\delta_2) \\ &= (x \pm y)\left(1 + \frac{x\delta_1 \pm y\delta_2}{x \pm y}\right). \end{aligned}$$

If the correct answer $x \pm y$ is very small relative to $|x|$ and $|y|$ (for example, when subtracting two nearly equal numbers), then there can be an arbitrarily large relative error in the result, compared to the errors in the initial $\bar{x}$ and $\bar{y}$. In particular, this relative error can be much larger than ϵ_{M}. This is called loss of significance (or catastrophic cancellation), and is a major cause of errors in floating-point calculations.

Example 1.5.2. To 4 s.f., the roots are

$$x_1 = 28 + \sqrt{783} = 55.98, \quad x_2 = 28 - \sqrt{783} = 0.01786.$$

However, working to 4 s.f. we would compute $\sqrt{783} = 27.98$, which would lead to the results

$$\bar{x}_1 = 55.98, \quad \bar{x}_2 = 0.02000.$$

The smaller root is not correct to 4 s.f., because of catastrophic cancellation. One way around this is to note that $x^2 - 56x + 1 = (x - x_1)(x - x_2)$, so the product of the roots equals the constant term: $x_1 x_2 = 1$. Computing $x_2 = 1/x_1 = 1/55.98 = 0.01786$ then gives the correct answer.

1.6 Symbolic Vs Numerical Representations

Symbolic computations require different data types from numerical ones; in MATLAB we can use the Symbolic Math Toolbox. In the following chapters, we will compare symbolic and numerical approaches to solving mathematical problems. One key difference is that symbolic computations are exact, but much more expensive to scale up to solve larger problems; for example, we will tackle numerical problems involving matrices of size $10^4 \times 10^4$ or larger, which would not be possible to successfully manipulate symbolically on a modern computer.

Symbolic computations are used to check or do laborious analytical work, but also to rigorously prove mathematical theorems which would be too onerous to carry out by hand. For instance, while the Lorenz equations were first noted to have strange dependencies on initial conditions, it was only in 2002 that this was mathematically proved to be a chaotic system.

Chapter 2

Continuous Functions

2.1 Interpolation

2.1.1 Motivation and Taylor Approximation

Interpolation is the process of constructing a function, usually a polynomial, that passes through a given set of data points. It is useful when a function is known only at finitely many points and we want to estimate its values elsewhere.

Remark 2.1.1 (Taylor Approximation vs Interpolation)**.** Taylor polynomials match the derivatives of a function at a single point. Interpolating polynomials match the values of a function at several distinct points. These are different constructions, and their error formulas are different.

Theorem 2.1.2 (Weierstrass Approximation Theorem)**.** If f is continuous on a closed interval $[a, b]$, then for every $\varepsilon > 0$, there exists a polynomial p such that

$$|f(x) - p(x)| < \varepsilon \qquad \text{for all } x \in [a, b].$$

Remark 2.1.3. The Weierstrass approximation theorem guarantees that a sufficiently accurate polynomial approximation exists. It does not guarantee that interpolating polynomials through arbitrary nodes converge as their degree increases.

2.1.2 Polynomial Interpolation

Suppose we are given $n + 1$ distinct nodes

$$x_0, x_1, \ldots, x_n$$

and corresponding function values

$$f(x_0), f(x_1), \ldots, f(x_n).$$

We seek a polynomial p_n of degree at most n satisfying

$$p_n(x_i) = f(x_i), \qquad i = 0, 1, \ldots, n.$$

Theorem 2.1.4 (Existence and Uniqueness of the Interpolating Polynomial)**.** Given $n + 1$ distinct nodes and prescribed function values, there exists a unique polynomial of degree at most n that interpolates the data.

Proof. Write

$$p_n(x) = a_0 + a_1 x + \cdots + a_n x^n.$$

The interpolation conditions give the linear system

$$\begin{pmatrix} 1 & x_0 & \cdots & x_0^n \\ 1 & x_1 & \cdots & x_1^n \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_n) \end{pmatrix}.$$

The determinant of the Vandermonde matrix is

$$\prod_{0 \le i < j \le n} (x_j - x_i),$$

which is non-zero because the nodes are distinct. Hence the system has a unique solution. □

2.1.3 Lagrange Interpolation

The Lagrange form constructs the interpolating polynomial directly from the interpolation nodes.

Definition 2.1.5 (Lagrange Basis Polynomials)**.** For distinct nodes $x_0, \ldots, x_n$, define

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}, \qquad i = 0, \ldots, n.$$

These satisfy

$$L_i(x_j) = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Theorem 2.1.6 (Lagrange Interpolation Formula)**.** The unique polynomial of degree at most n interpolating f at $x_0, \ldots, x_n$ is

$$\boxed{p_n(x) = \sum_{i=0}^{n} f(x_i) L_i(x).}$$

Example 2.1.7. Find the quadratic interpolating polynomial for

$$f(x) = \cos x$$

at the nodes

$$x_0 = -\frac{\pi}{4}, \qquad x_1 = 0, \qquad x_2 = \frac{\pi}{4}.$$

The corresponding function values are

$$f(x_0) = \frac{1}{\sqrt{2}}, \qquad f(x_1) = 1, \qquad f(x_2) = \frac{1}{\sqrt{2}}.$$

By symmetry, the interpolating polynomial is even, so write

$$p_2(x) = 1 + cx^2.$$

Using $p_2(\pi/4) = 1/\sqrt{2}$,

$$1 + c\frac{\pi^2}{16} = \frac{1}{\sqrt{2}}.$$

Therefore,

$$\boxed{p_2(x) = 1 + \frac{16}{\pi^2}\left(\frac{1}{\sqrt{2}} - 1\right) x^2.}$$

Remark 2.1.8 (Practical Use)**.** For hand calculations, the Lagrange formula is often convenient because it gives the polynomial explicitly. For repeated interpolation with changing data, other forms such as Newton interpolation may be more efficient.

2.1.4 Interpolation Error

The interpolation polynomial generally differs from the original function between the interpolation nodes.

Theorem 2.1.9 (Lagrange Interpolation Error)**.** Suppose $f \in C^{n+1}([a,b])$, and let p_n interpolate f at the distinct nodes $x_0, \dots, x_n \in [a,b]$. For each $x \in [a,b]$, there exists a point $\xi_x \in [a,b]$ such that

$$\boxed{f(x) - p_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \prod_{i=0}^{n} (x - x_i).}$$

If

$$|f^{(n+1)}(x)| \le M \qquad \text{for all } x \in [a,b],$$

then

$$\boxed{|f(x) - p_n(x)| \le \frac{M}{(n+1)!} \left| \prod_{i=0}^{n} (x - x_i) \right|.}$$

Example 2.1.10. Suppose $f(x) = \cos x$ is interpolated at

$$x_0 = -\frac{\pi}{4}, \qquad x_1 = 0, \qquad x_2 = \frac{\pi}{4}.$$

Since

$$f^{(3)}(x) = \sin x,$$

we have $|f^{(3)}(x)| \le 1$. Therefore,

$$|f(x) - p_2(x)| \le \frac{1}{6} \left| \left(x + \frac{\pi}{4}\right) x \left(x - \frac{\pi}{4}\right) \right|.$$

On $[-\pi/4, \pi/4]$,

$$\left| \left(x + \frac{\pi}{4}\right) x \left(x - \frac{\pi}{4}\right) \right| = \left| x \left(x^2 - \frac{\pi^2}{16}\right) \right|.$$

A simple, though not necessarily sharp, bound is

$$|x| \le \frac{\pi}{4}, \qquad \left| x^2 - \frac{\pi^2}{16} \right| \le \frac{\pi^2}{16}.$$

Hence

$$\boxed{|f(x) - p_2(x)| \le \frac{\pi^3}{384}.}$$

Remark 2.1.11 (How to Use the Error Formula)**.** (i) Identify the interpolation degree n.

(ii) Differentiate the function $n+1$ times.

(iii) Find an upper bound M for $|f^{(n+1)}|$ on the interval.

(iv) Bound the product $|\prod_{i=0}^{n}(x - x_i)|$.

(v) Substitute both bounds into the error estimate.

2.1.5 Runge Phenomenon and Chebyshev Nodes

Increasing the degree of an interpolating polynomial does not necessarily improve the approximation everywhere.

Remark 2.1.12 (Runge Phenomenon)**.** Interpolation at equally spaced nodes can produce large oscillations near the endpoints as the polynomial degree increases. This behaviour is known as the Runge phenomenon.

A common remedy is to choose Chebyshev nodes, which cluster near the endpoints of the interpolation interval.

Definition 2.1.13 (Chebyshev Nodes)**.** The $n+1$ Chebyshev nodes on $[-1,1]$ are

$$\boxed{x_k = \cos\left(\frac{(2k+1)\pi}{2(n+1)}\right), \qquad k = 0, \dots, n.}$$

For these nodes, the nodal polynomial satisfies

$$\boxed{\max_{x\in[-1,1]} \left|\prod_{k=0}^{n}(x - x_k)\right| = 2^{-n}.}$$

Consequently, if

$$|f^{(n+1)}(x)| \le M \qquad \text{on } [-1,1],$$

then

$$\boxed{\max_{x\in[-1,1]} |f(x) - p_n(x)| \le \frac{M}{(n+1)!2^n}.}$$

Remark 2.1.14. The error bound depends on both the node locations and the size of the $(n+1)$-st derivative. Chebyshev nodes control the nodal-product factor, but they do not guarantee small error for every function and every degree.

2.2 Non-Linear Equations

2.2.1 Problem and Motivation

We want to find a solution to

$$f(x) = 0.$$

Such a solution is called a root or zero of f.

For many nonlinear functions, an exact solution is unavailable or impractical to calculate. Numerical methods therefore generate a sequence of approximations

$$x_0, x_1, x_2, \dots$$

intended to converge to a root r.

Different methods trade off reliability, convergence speed, computational cost and the amount of information required about f.

2.2.2 Bisection Method

The bisection method repeatedly halves an interval known to contain a root.

Theorem 2.2.1 (Bisection Method)**.** Suppose f is continuous on $[a_0, b_0]$ and

$$f(a_0)f(b_0) < 0.$$

Then f has at least one root in (a_0, b_0). Repeated bisection produces nested intervals containing a root, with interval lengths tending to zero.

At each iteration, calculate

$$m_k = \frac{a_k + b_k}{2}.$$

If $f(m_k) = 0$, the root has been found exactly. Otherwise, retain the half-interval on which the function changes sign:

$$[a_{k+1}, b_{k+1}] = \begin{cases} [a_k, m_k], & f(a_k)f(m_k) < 0, \\ [m_k, b_k], & f(m_k)f(b_k) < 0. \end{cases}$$

Theorem 2.2.2 (Bisection Error Bound)**.** After n bisection steps, the interval length is

$$b_n - a_n = \frac{b_0 - a_0}{2^n}.$$

If $m_n = (a_n + b_n)/2$ is the midpoint of the final interval, then the root r satisfies

$$\boxed{|r - m_n| \leq \frac{b_0 - a_0}{2^{n+1}}.}$$

To guarantee midpoint error at most ε, it is sufficient to choose n such that

$$\frac{b_0 - a_0}{2^{n+1}} \leq \varepsilon.$$

Equivalently,

$$\boxed{n \geq \log_2\left(\frac{b_0 - a_0}{2\varepsilon}\right).}$$

Round up to the next integer if necessary.

Remark 2.2.3 (Advantages and Disadvantages)**.**

- Guaranteed convergence when continuity and a sign-changing initial interval are satisfied.
- The error bound is explicit.
- Convergence is linear, with the interval length halved at every step.
- It does not use derivative information.
- It can be slower than Newton's method or the secant method.

2.2.3 Fixed-Point Iteration

A root-finding problem can sometimes be rewritten in the form

$$x = g(x).$$

A solution of this equation is called a fixed point of g.

The fixed-point iteration is

$$\boxed{x_{n+1} = g(x_n).}$$

Remark 2.2.4. The choice of rearrangement matters. Different functions g can have the same fixed point but very different convergence behaviour.

Theorem 2.2.5 (Contraction Mapping Theorem)**.** Suppose $g : [a, b] \to [a, b]$ and there exists a constant $L < 1$ such that

$$|g(x) - g(y)| \leq L|x - y| \qquad \text{for all } x, y \in [a, b].$$

Then g has a unique fixed point $r \in [a, b]$, and the iteration

$$x_{n+1} = g(x_n)$$

converges to r for every initial value $x_0 \in [a, b]$.

A useful sufficient condition for contraction is

$$|g'(x)| \leq L < 1 \qquad \text{for all } x \in [a, b],$$

provided g maps $[a, b]$ into itself.

Theorem 2.2.6 (A Priori Error Bound)**.** Under the contraction assumptions,

$$\boxed{|x_n - r| \leq \frac{L^n}{1-L}|x_1 - x_0|.}$$

Remark 2.2.7 (Local Convergence)**.** If $g(r) = r$ and $|g'(r)| < 1$, then the fixed point is locally attractive: iterations starting sufficiently close to r converge to it, provided the iteration remains in a region where the contraction condition holds. This local condition is weaker than requiring a contraction on the entire interval.

2.2.4 Order of Convergence

Let $x_n \to r$, where r is the exact solution, and define the error

$$e_n = x_n - r.$$

Definition 2.2.8 (Order of Convergence)**.** The sequence (x_n) converges to r with order $p \geq 1$ if there exists a constant $C > 0$ such that

$$\boxed{\lim_{n\to\infty} \frac{|e_{n+1}|}{|e_n|^p} = C.}$$

Equivalently, asymptotically,

$$|e_{n+1}| \approx C|e_n|^p.$$

- $p = 1$: linear convergence.
- $p > 1$: superlinear convergence.
- $p = 2$: quadratic convergence.

Remark 2.2.9. Linear convergence reduces the error by approximately a constant factor per iteration. Quadratic convergence can produce a dramatic reduction in error once the iterates are sufficiently close to the root.

2.2.5 Newton's Method

Newton's method uses the tangent line to approximate a nonlinear function near the current iterate.

The tangent line to f at x_n is

$$y = f(x_n) + f'(x_n)(x - x_n).$$

Setting $y = 0$ and solving for x gives Newton's iteration.

Definition 2.2.10 (Newton's Method)**.** Given an initial guess x_0, define

$$\boxed{x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.}$$

The method requires $f'(x_n) \neq 0$ at each iteration.

Theorem 2.2.11 (Local Quadratic Convergence of Newton's Method)**.** Suppose $f(r) = 0$, $f'(r) \neq 0$, and f is sufficiently smooth near r. If x_0 is sufficiently close to r, then Newton's method converges locally to r, with quadratic convergence under the standard regularity assumptions.

For a simple root, the error satisfies asymptotically

$$\boxed{e_{n+1} \approx \frac{f''(r)}{2f'(r)} e_n^2.}$$

Example 2.2.12. Apply Newton's method to solve

$$f(x) = x^2 - 2 = 0.$$

Since $f'(x) = 2x$, the iteration is

$$x_{n+1} = x_n - \frac{x_n^2 - 2}{2x_n} = \frac{1}{2}\left(x_n + \frac{2}{x_n}\right).$$

Starting with $x_0 = 1$,

$$x_1 = 1.5, \qquad x_2 \approx 1.416667, \qquad x_3 \approx 1.414216.$$

The iterates converge to $\sqrt{2}$.

Remark 2.2.13 (Advantages and Disadvantages).
- Usually very fast near a simple root.
- Has quadratic local convergence under suitable conditions.
- Requires evaluation of the derivative.
- Can fail if $f'(x_n) = 0$ or becomes very small.
- Convergence is not guaranteed for an arbitrary initial guess.

2.2.6 Newton's Method for Systems

For a system of m nonlinear equations in m unknowns, write

$$\mathbf{F}(\mathbf{x}) = \mathbf{0},$$

where

$$\mathbf{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_m \end{pmatrix}, \qquad \mathbf{F}(\mathbf{x}) = \begin{pmatrix} F_1(\mathbf{x}) \\ \vdots \\ F_m(\mathbf{x}) \end{pmatrix}.$$

The Jacobian matrix is

$$J_{\mathbf{F}}(\mathbf{x}) = \begin{pmatrix} \dfrac{\partial F_1}{\partial x_1} & \cdots & \dfrac{\partial F_1}{\partial x_m} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial F_m}{\partial x_1} & \cdots & \dfrac{\partial F_m}{\partial x_m} \end{pmatrix}.$$

Definition 2.2.14 (Newton's Method for Systems). At each iteration, solve

$$J_{\mathbf{F}}(\mathbf{x}_n)\Delta\mathbf{x}_n = -\mathbf{F}(\mathbf{x}_n)$$

for $\Delta\mathbf{x}_n$, then update

$$\boxed{\mathbf{x}_{n+1} = \mathbf{x}_n + \Delta\mathbf{x}_n.}$$

Remark 2.2.15. In numerical computation, solve the linear system for $\Delta\mathbf{x}_n$ rather than explicitly calculating $J_{\mathbf{F}}(\mathbf{x}_n)^{-1}$. The Jacobian must be nonsingular at the solution for the standard local convergence result to apply.

Example 2.2.16. In MATLAB, a Newton step for a system can be implemented as follows:

```
delta = -J(x) \ F(x);
x_new = x + delta;
```

Here, `F(x)` returns the vector of function values and `J(x)` returns the Jacobian matrix. The backslash operator solves the linear system.

2.2.7 Secant Method

The secant method approximates the derivative in Newton's method using a finite difference:

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}.$$

Substituting this into Newton's iteration gives the secant method.

Definition 2.2.17 (Secant Method)**.** Given two initial approximations x_0 and x_1, define

$$\boxed{x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}.}$$

The denominator must be non-zero.

Remark 2.2.18 (Convergence)**.** Under suitable regularity assumptions and when the iterates approach a simple root, the secant method has order of convergence

$$\boxed{p = \frac{1+\sqrt{5}}{2} \approx 1.618.}$$

This is superlinear convergence: faster than linear convergence, but slower than the quadratic convergence of Newton's method.

Remark 2.2.19 (Advantages and Disadvantages)**.**

- Does not require an explicit derivative.
- Often converges faster than bisection.
- Usually has lower derivative-evaluation cost than Newton's method.
- Requires two initial approximations.
- Does not have the same unconditional interval guarantee as bisection.

2.2.8 Comparison of Root-Finding Methods

Method	Derivative	Convergence order	Reliability
Bisection	No	Linear	Guaranteed with sign change
Fixed-point	No	Depends on g	Depends on contraction
Newton	Yes	Quadratic locally	Initial guess matters
Secant	No	1.618 locally	Initial guesses matter

2.2.9 Problem-Solving Checklist

Remark 2.2.20 (Choosing and Applying a Method)**.**

(i) **Bisection:** check continuity and a sign change at the endpoints.

(ii) **Fixed-point iteration:** rearrange the equation as $x = g(x)$, then investigate whether g maps the interval into itself and whether $|g'(x)| < 1$.

(iii) **Newton's method:** calculate f', check that the derivative is non-zero at the iterates, and use a suitable initial guess.

(iv) **Secant method:** use two initial guesses and check that $f(x_n) - f(x_{n-1}) \neq 0$.

(v) **Error estimates:** identify the theorem's hypotheses before applying its bound.

(vi) **Convergence order:** distinguish a guaranteed convergence result from a local or asymptotic convergence result.